SISTEMAS DE GESTIÓN EMPRESARIAL (2º DAM) · OPERACIÓN BÓVEDA

Resumen de Fundamentos de Python

Síntesis técnica extraída de la Sesión 1: Primeros pasos con Python

Este resumen recoge exclusivamente los conceptos de programación, sintaxis, estructuras de datos y manejo de ficheros en **Python** presentados en la Sesión 1, omitiendo la narrativa del ejercicio y centrándose en su equivalencia práctica frente a Java.

1. Estructura básica y ejecución (Lección 01)

Python destaca por su sintaxis minimalista frente a Java: no requiere declarar una clase contenedora, ni un método `main`, ni puntos y comas al final de cada instrucción, ni fase previa de compilación manual.

- **Salida por consola:** Se utiliza directamente la función `print("Hola")`.
- **Ejecución desde terminal:** Se lanza mediante el intérprete con `python hola.py` (o `py hola.py` en entornos Windows).
- **Estructura de scripts de la práctica:** El proyecto se divide en dos archivos situados en la misma carpeta: `datos.py` (almacena las estructuras de datos y no debe modificarse) y `mision.py` (script ejecutable donde se programa la lógica).

2. Variables, tipado dinámico y entrada/salida (Lección 02)

En Python las variables no tienen un tipo fijo declarado; el tipo pertenece al valor asignado en cada momento y puede cambiar dinámicamente durante la ejecución.

Tipo / Función	Ejemplo en Python	Observaciones clave
`str` (Cadena)	`agente = "Vega"`	Texto delimitado por comillas dobles o simples.
`int` (Entero)	`nivel = 3`	Puede reasignarse a otro tipo (ej. `nivel = "tres"`) sin error.
`float` (Decimal)	`salario = 3200.50`	Separador decimal mediante punto.
`bool` (Booleano)	`activo = True`	Valores `True` y `False` siempre con inicial mayúscula.
`type()`	`print(type(nivel))`	Devuelve el tipo actual del dato almacenado (ej. `<class 'int'>`).
`input()`	`edad = int(input("Edad: "))`	Captura datos por teclado siempre como texto (`str`); requiere conversión explícita (`int()`) para operar numéricamente.

Tipo / Función	Ejemplo en Python	Observaciones clave
f-strings	`f"Agente {agente}, nivel {nivel}"`	Permite interpolar variables directamente dentro de una cadena de texto mediante llaves.

3. Condicionales y bloques por sangría (Lección 03)

Python prescinde de las llaves `{ }` y de los paréntesis obligatorios en las condiciones. Los bloques de código se abren con dos puntos (`:`) y se delimitan estrictamente mediante **sangría de 4 espacios** (una indentación incorrecta provoca un error de ejecución).

- **Palabras clave:** `if` , `elif` (equivale a `else if` en Java) y `else` .
- **Operadores lógicos:** Se escriben con palabras en inglés (`and` , `or` , `not`) en lugar de `&&` , `||` y `!` .
- **Métodos de cadena y comparación:** `ip.startswith("10.")` comprueba el prefijo y `==` compara igualdad de valores directamente.

```
if ip.startswith("10."):
    print("Interna")
elif ip == "":
    print("Vacía")
else:
    print("Externa")
```

4. Bucles: `for` y `while` (Lección 04)

El bucle `for` en Python funciona siempre como un *for-each* que recorre directamente los elementos de una secuencia o colección, sin necesidad de índices manuales.

- **Iteración sobre colecciones:** `for e in empleados:` recorre cada elemento directamente.
- **Iteración numérica con `range()` :** Cuando se requiere un contador numérico, `for i in range(3):` genera la secuencia de enteros `0, 1, 2` .
- **Bucle condicional:** `while intentos > 0:` mantiene la sintaxis habitual con dos puntos y sangría.

5. Listas, tuplas y rebanado o *slicing* (Lección 05)

Python ofrece dos estructuras secuenciales fundamentales cuya diferencia principal radica en la mutabilidad de sus elementos:

Característica	Listas (`list`)	Tuplas (`tuple`)
Delimitador	Corchetes: `["10.0.0.15", "10.0.0.22"]`	Paréntesis: `(6, "2026-10-15 03:17", "185.220.101.47")`
Mutabilidad	**Mutable:** permite añadir (`ips.append(...)`) o modificar elementos.	**Inmutable:** no se puede modificar una vez creada.
Operaciones comunes	`len(ips)` (longitud), `ips[0]` (primero), `ips[-1]` (último), `"10.0.0.22" in ips` (pertenencia).	**Desempaquetado:** `id_emp, fecha, ip = acceso` asigna cada posición a una variable en una línea.

Rebanado (*slicing*) y comparación de textos: Las cadenas se pueden recortar igual que las listas indicando `[inicio:fin]`. Por ejemplo, sobre `fecha = "2026-10-15 03:17"`, la expresión `fecha[11:16]` extrae `"03:17"`. Además, las horas en formato `HH:MM` pueden compararse alfabéticamente con operadores estándar (`"03:17" < "08:00"` devuelve `True`).

6. Diccionarios e importación de módulos (Lección 06)

Los diccionarios (`dict`) equivalen a un `Map<String, Object>` o `HashMap` en Java. Almacenan pares `clave: valor` entre llaves `{ }` y se leen o modifican mediante corchetes con el nombre de la clave.

- **Importación entre archivos:** La sentencia `from datos import empleados` importa una variable o estructura definida en el fichero `datos.py`.
- **Tablas en memoria:** Una lista de diccionarios representa una tabla de registros (cada diccionario es una fila).
- **Agregación rápida:** La función integrada `sum()` combinada con una expresión generadora permite sumar campos de todos los registros en una sola línea.

```
from datos import empleados

e = {"id": 6, "nombre": "Raúl Quintero", "depto": "Ventas", "salario": 2950}
print(e["nombre"])     # Acceso por clave
e["salario"] = 3000    # Modificación de valor

for e in empleados:
    if e["depto"] == "Sistemas":
        print(e["nombre"], e["salario"])

print(sum(e["salario"] for e in empleados))
```

Síntesis de diferencias clave Java frente a Python: Sin llaves ni punto y coma (bloques por sangría de 4 espacios); tipado dinámico sin declaración previa; `elif` en lugar de `else if`; bucles `for` siempre orientados a colecciones o `range()`; y estructuras nativas directas mediante `[]` (listas), `()` (tuplas) y `{}` (diccionarios).

7. Guía de resolución de las actividades (Misiones 1 a 7)

En la plataforma **RootTheBox** (*Operación Bóveda*), cada misión aplica directamente los conceptos anteriores sobre las dos estructuras definidas en `datos.py`: la lista de diccionarios `empleados` y la lista de tuplas `accesos` (cada tupla tiene el formato `(id_emp, fecha, ip)`).

Misión	Concepto aplicado	Explicación y resolución en Python
Misión 1 Sin etiquetas	Tipado dinámico y `type()` (Lección 02)	En Python una variable no tiene un tipo fijo: primero apunta a un entero (`7`) y después se reasigna a una cadena de texto (`"siete"`). Al ejecutar `print(type(x))` se evalúa el tipo del último valor almacenado, imprimiendo `<class 'str'>`.
Misión 2 La nómina fantasma	Listas de diccionarios y `sum()` (Lección 06)	Cada elemento `e` de `empleados` es un diccionario con la clave `"salario"`. Para calcular el gasto total mensual sin bucles largos se usa una expresión generadora dentro de `sum()`: `print(sum(e["salario"] for e in empleados))`
Misión 3 El intruso	Tuplas, condicionales y búsqueda cruzada (Lecciones 03–06)	Se recorre `accesos` desempaquetando cada tupla en `id_emp, fecha, ip`. Si `not ip.startswith("10.")`, es un acceso externo. Con ese `id_emp` se recorre `empleados` buscando coincidencia con `e["id"] == id_emp` para obtener su nombre y formatearlo en minúsculas, sin tildes y con guion bajo.
Misión 4 Rastro digital	Filtrado de cadenas con `startswith()` (Lección 03)	En el mismo recorrido sobre `accesos` empleado en la Misión 3, basta con imprimir la variable `ip` cuando no empiece por `"10."`: `if not ip.startswith("10."): print(ip)`
Misión 5 Hora cero	Orden cronológico en listas (Lección 05)	La lista `accesos` almacena las tuplas cronológicamente y el campo `fecha` tiene el formato `"AAAA-MM-DD HH:MM"`. Al filtrar los accesos de la IP externa, el último registro coincidente corresponde al acceso más reciente, entregando directamente su cadena `fecha` completa.
Misión 6 La caja fuerte	Filtrado por clave, media y `round()` (Lecciones 04 y 06)	Se filtran los empleados donde `e["depto"] == "Sistemas"`, acumulando sus salarios y contando cuántos son (o guardándolos en una lista auxiliar `salarios`). La media se obtiene con `sum(salarios) / len(salarios)` y se redondea al entero más cercano mediante `round(...)`.
Misión 7 Fuera de horario	*Slicing* de cadenas y operadores lógicos (Lecciones 03 y 05)	De cada tupla de `accesos` se extrae la hora con `hora = fecha[11:16]`. Como el formato `HH:MM` permite comparación alfabética directa, se incrementa un contador cuando se cumple la condición: `if hora < "08:00" or hora >= "22:00":`

```python
# Plantilla de resolución completa para mision.py
from datos import empleados, accesos

# Misión-1
x = 7
x = "siete"
print("Misión 1:", type(x))

# Misión-2
print("Misión 2:", sum(e["salario"] for e in empleados))

# Misión-3
for id_empleado, fecha, ip in accesos:
    if not ip.startswith("10."):
        for e in empleados:
            if e["id"] == id_empleado:
                print("Misión 3:", e["nombre"].lower().replace(" ", "_"))

# Misión-4
for id_empleado, fecha, ip in accesos:
    if not ip.startswith("10."):
        print("Misión 4:", ip)

# Misión-5
for id_empleado, fecha, ip in accesos:
    if ip == "185.220.101.47":
        print("Misión 5:", fecha)

# Misión-6
suma = 0
contador = 0
for e in empleados:
    if e["depto"] == "Sistemas":
        suma += e["salario"]
        contador += 1
print("Misión 6:", round(suma / contador))

# Misión-7
fuera = 0
for id_empleado, fecha, ip in accesos:
    hora = fecha[11:16]
    if hora < "08:00" or hora >= "22:00":
        fuera += 1
print("Misión 7:", fuera)
```